딥러닝

AI 추론 최적화

(주)소이넷 인턴십 AI 추론 엔진 최적화 프로젝트 상세 소개

김문수

프로젝트 요약

오픈소스 모델을 최적화된 추론 엔진으로 전환

실시간 서비스의 처리 속도, GPU 메모리, 호환성과 검증 요구사항을 고려하여 추론 파이프라인을 재구성했습니다.

수행 범위

7개 모델

비전 및 자연어 처리 모델

핵심 개발

CUDA + CFG

GPU 전처리, 후처리 및
레이어 설정

검증 결과

FPS 2.3배

YOLOX-X 벤치마크 성능 개선

GPU 메모리

800MB

YOLOX-X 기준 1,800MB에서 절감

주요 수행 업무

PyTorch 입출력 파이프라인과 모델 실행 경로 분석 및 디버깅

C++/CUDA 기반 전처리 및 후처리 모듈 구현

연산 그래프를 엔진 전용 레이어 설정 파일로 변환

개선 효과

실시간 AI 서비스의 지연 시간 감소

GPU 메모리 사용량과 서비스 운영 비용 절감

출력 정확도를 유지하며 연구용 코드를 추론 엔진으로 이전

개발 프로세스

모델 분석부터 성능 검증까지의 최적화 과정

오픈소스 모델 분석부터 엔진용 구현과 벤치마크 측정까지 전 과정을 수행했습니다.

01 모델 확보

오픈소스 모델 코드
다운로드 및 분석

02 입력 파이프라인

전처리 및 텐서 데이터 배치 분석

03 GPU 모듈 구현

C++/CUDA 전처리 및
커스텀 레이어 구현

04 그래프 분석

DAG 기반 모델 아키텍처 시각화

05 엔진 설정

레이어별 .cfg 파일 작성
및 출력 검증

06 벤치마크

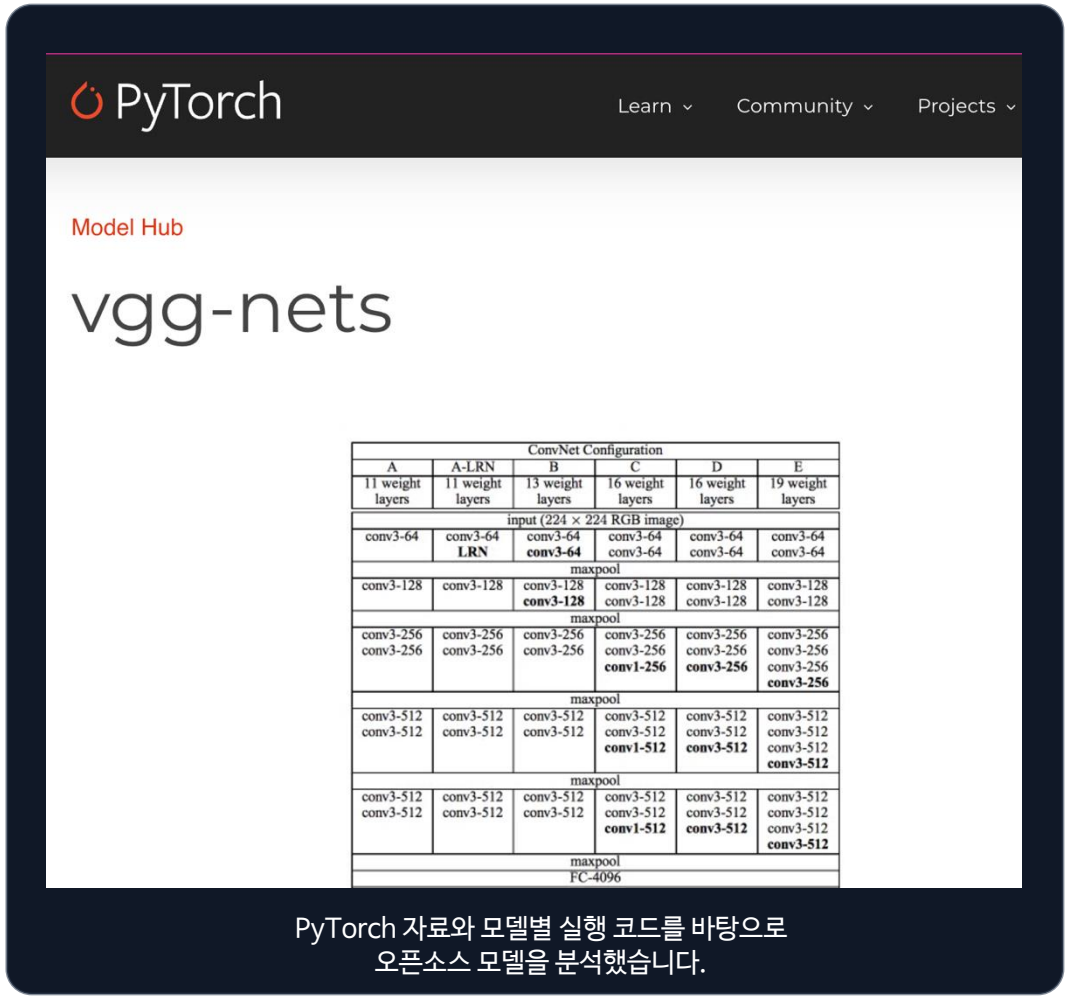
최적화 후 FPS 및
GPU 메모리 측정

핵심 원칙: 레이어별 호환성을 조기에 확인해 수치적, 구조적 불일치가 누적되지 않도록 검증했습니다.

적용 모델

다양한 모델 계열

분류, 이미지 분할, 객체 탐지, 자세 추정 및 자연어 처리 모델을 다뤘습니다.



ConvNet Configuration

A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					

PyTorch 자료와 모델별 실행 코드를 바탕으로
오픈소스 모델을 분석했습니다.

이미지 분류

VGG16 / VGG19
EfficientNet-B6

이미지 분할

FCN

객체 탐지

YOLOX-X

자세 추정

PoseNet Transpose-h-a4

언어 모델

BART Paragraph

입력 전처리를 GPU 친화적인 모듈로 구현

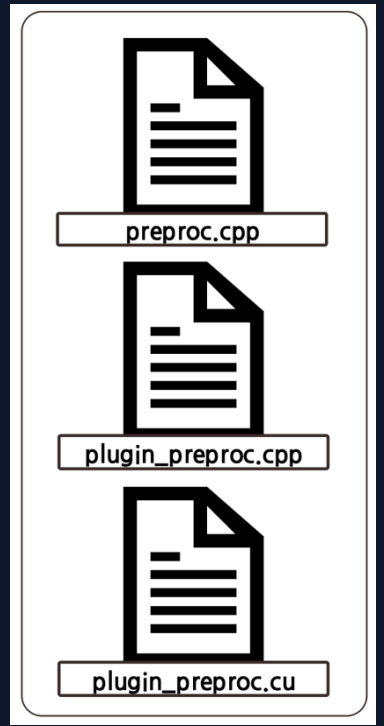
모델별 입력 파이프라인을 디버깅하고 반복되는 전처리 연산을 C++/CUDA 모듈로 재구현했습니다.

```python
input_image = Image.open(filename)  input_image: <PIL.JpegImagePlugin.JpegImageFile
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
input_tensor = preprocess(input_image)
input_batch = input_tensor.unsqueeze(0) # create a mini-batch as expected by the mode

# move the input and model to GPU for speed if available
if torch.cuda.is_available():
    input_batch = input_batch.to('cuda')
    model.to('cuda')

with torch.no_grad():
```

PyTorch의 변환 과정을
네이티브 모듈로 구현했습니다.



preproc.cpp
plugin_preproc.cpp
plugin_preproc.cu

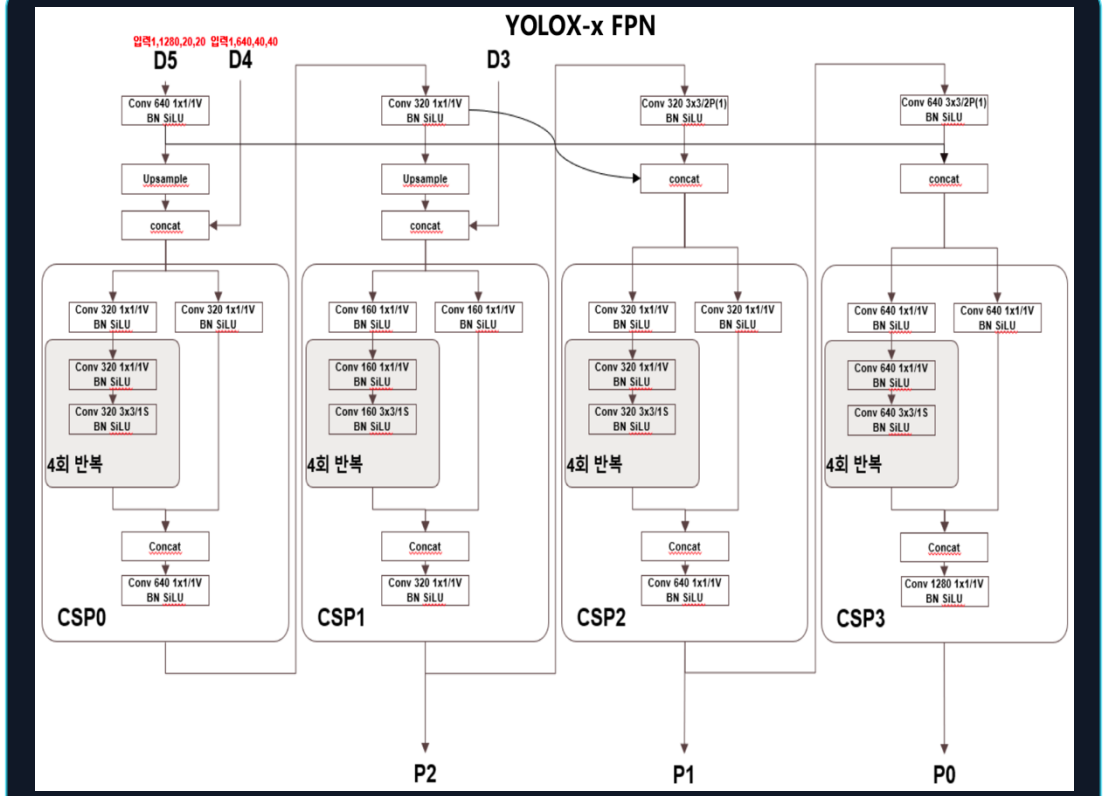
주요 구현 연산

- 크기 조정
- BGR → RGB 변환
- 자르기
- 정규화
- 패딩

결과: Python 처리 부담을 줄이고
최적화 엔진으로 연결되는
경로를 단순화했습니다.

모델 실행 흐름을 레이어 단위 명세로 변환

소스 코드를 분석해 모델 DAG를 재구성하고 엔진과 호환되는 설정 파일로 변환했습니다.



YOLOX-X FPN DAG: 연산 경로, 레이어 특성, 텐서 흐름

bot-sort-yolox-x.cfg | bot-sort-baseline.cfg | bot-sort.cpp | nms.cpp

```
[conv2d] input=P2 channel=320 refname=STEM0
[conv2d] input=P1 channel=320 refname=STEM1
[conv2d] input=P0 channel=320 refname=STEM2
## CLS conv ##
[conv2d] input=STEM0 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
[conv2d] input=STEM1 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
[conv2d] input=STEM2 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
## REG conv ##
[conv2d] input=STEM0 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
[conv2d] input=STEM1 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
[conv2d] input=STEM2 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t

## CLS PREDS ##
[conv2d] input=CLS_0 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_0
[conv2d] input=CLS_1 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_1
[conv2d] input=CLS_2 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_2
## REG PREDS ##
[conv2d] input=REG_0 channel=4 weight_order=wa act=identity refname=REGP_0
[conv2d] input=REG_1 channel=4 weight_order=wa act=identity refname=REGP_1
[conv2d] input=REG_2 channel=4 weight_order=wa act=identity refname=REGP_2
## OBJ PREDS ##
[conv2d] input=REG_0 channel=1 weight_order=wa act=sigmoid refname=OBJP_0
[conv2d] input=REG_1 channel=1 weight_order=wa act=sigmoid refname=OBJP_1
[conv2d] input=REG_2 channel=1 weight_order=wa act=sigmoid refname=OBJP_2

[concat] input=REGP_0,OBJP_0,CLSP_0 axis=1 [reshape] shape=$BATCH_SIZE,$CLASS_COUNT+5,-1
```

레이어별 .cfg 파일 작성 및
원본 모델과의 즉각적인 호환성 검증

모델 출력 이후까지 전체 추론 파이프라인 구현

네트워크 연산 최적화에 이어 모델별 출력 디코딩 로직도 구현했습니다.

```cpp
__global__ void kernel_yolox_postproc(float* output, float* input, float gain, int tcount)
{
    int pos = threadIdx.x + blockIdx.x * blockDim.x;
    if (pos >= tcount) return;

    int base = pos * 6;
    float x1 = input[base + 0];
    float y1 = input[base + 1];
    float x2 = input[base + 2];
    float y2 = input[base + 3];

    x1 /= gain;
    y1 /= gain;
    x2 /= gain;
    y2 /= gain;

    output[base + 0] = (float)((int)x1);
    output[base + 1] = (float)((int)y1);
    output[base + 2] = (float)((int)x2);
    output[base + 3] = (float)((int)y2);

    output[base + 4] = input[base + 4];
    output[base + 5] = input[base + 5];
}


void cuda_yolox_postproc(float* output, float* input, float gain, int N, int bbox_count, cudaStream_t cudaStream)
{
    int tcount = N * bbox_count;
    int block = 64;
    int grid = (tcount - 1) / block + 1;

    kernel_yolox_postproc << <grid, block, 0, cudaStream >> > (output, input, gain, tcount);
}
```

예시: YOLOX CUDA 후처리 커널 및 호스트 래퍼

출력 텐서 분석

모델 출력을 디버깅해 텐서 배치, 점수 인코딩과
작업별 디코딩 방식을 파악했습니다.

네이티브 모듈 및 후처리 레이어

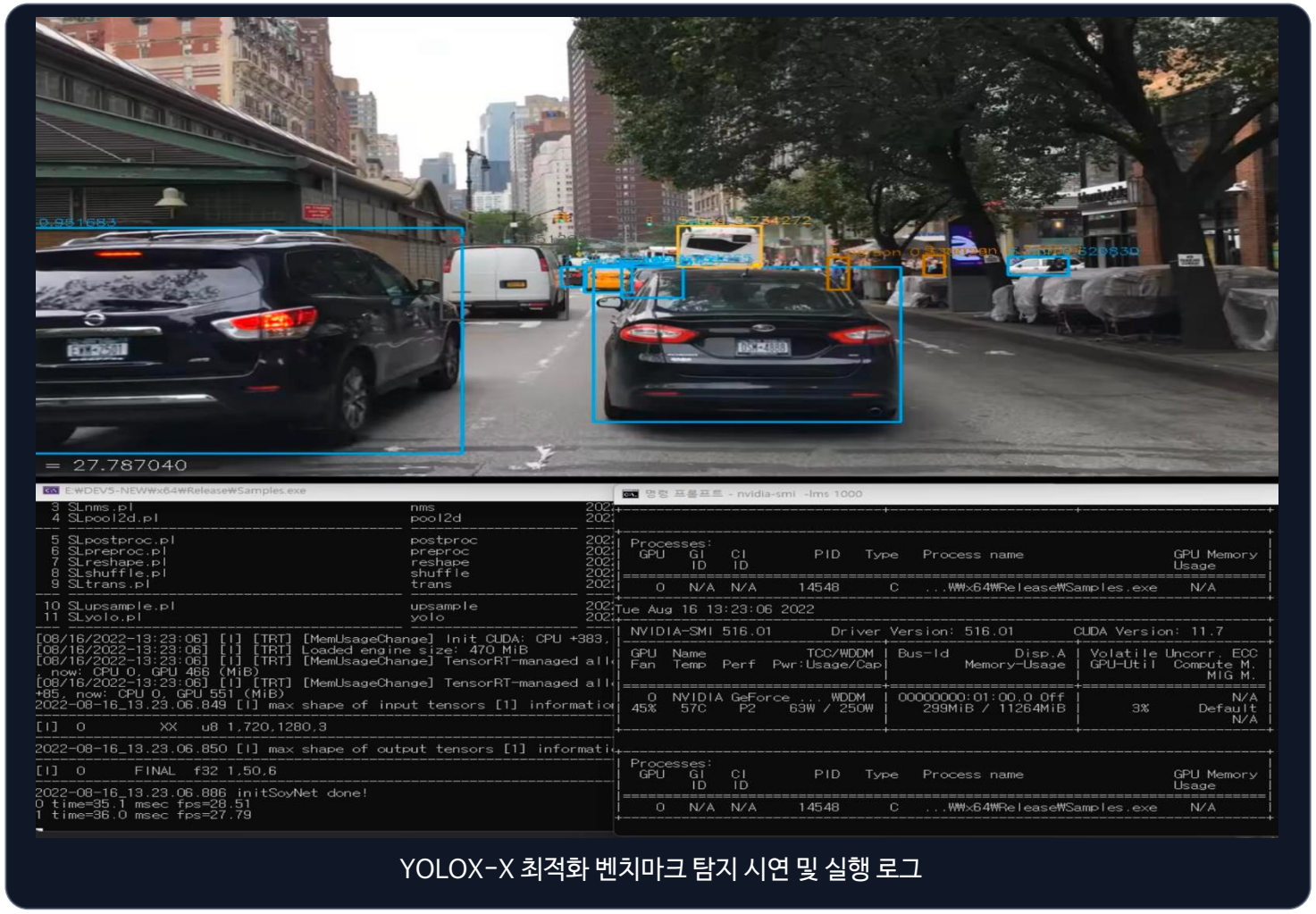
탐지, 분할 및 분류 후처리를
C++/CUDA로 구현했습니다.

추론 엔진 통합

여러 모델 계열에서 재사용할 수 있도록
최적화 파이프라인을 모듈화했습니다.

YOLOX-X: 추론 속도 향상 및 GPU 메모리 절감

원본 모델과 일치하는 출력을 유지하며 추론 처리량을 높이고 GPU 메모리 사용량을 줄였습니다.



YOLOX-X 최적화 벤치마크 탐지 시연 및 실행 로그

FPS

16-17 -> 38-39

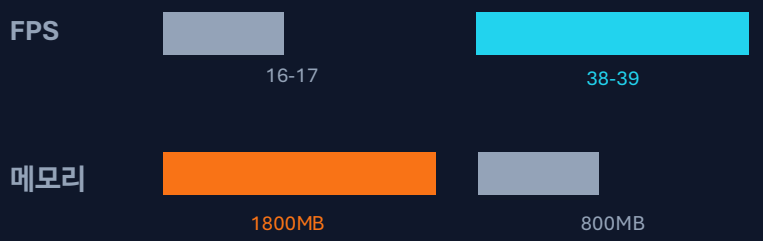
약 2.3배 향상

GPU 메모리

1800 -> 800MB

약 55.6% 감소

최적화 전후 비교



핵심 역량

인턴십 프로젝트에서 검증한 역량

실제 서비스에 적용할 AI 모델의 실행 효율, 이식성 및 성능을 직접 개선하고 측정한 프로젝트입니다.

모델 수준 디버깅

오픈소스 모델 코드, 텐서 흐름과
전처리 로직을 분석하고 디버깅했습니다.

시스템 구현

C++/CUDA 모듈과 커스텀 레이어를
구현해 최적화된 런타임에 적용했습니다.

단계별 검증

원본과 레이어별 출력을 비교하며
결과 정확도를 확인했습니다.

성능 측정 및 개선

FPS와 GPU 메모리를 측정하여
최적화 효과를 수치로 확인했습니다.